

Tema 3.7: Pruebas Unitarias en Back-End con Spring Boot, Maven, JUnit Jupiter y Mockito

Resumen

Este tema aborda los principios fundamentales del diseño y ejecución de pruebas unitarias de software en la capa back-end utilizando la herramienta de construcción **Apache Maven**, el motor de pruebas **JUnit 5 (JUnit Jupiter)** y la librería de simulación (*mocking*) **Mockito 5.x** en aplicaciones Spring Boot 3.x. Se analiza la integración de dependencias en el archivo `pom.xml`, el plugin `maven-surefire-plugin`, la arquitectura interna modular de JUnit 5, las anotaciones y aserciones avanzadas de JUnit Jupiter, la estrategia de aislamiento mediante dobles de prueba (*mocks*), el patrón Arrange-Act-Assert (AAA) y la ejecución de pruebas de controladores REST con `MockMvc` y `@WebMvcTest`.

Concepto Principal (El 'Qué')

En la ingeniería de software profesional, una **Prueba Unitaria** (*Unit Test*) es un bloque de código automatizado que verifica la validez del comportamiento de la unidad lógica más pequeña e indivisible de una aplicación (generalmente un método de una clase) de forma totalmente aislada de sus dependencias externas, tales como bases de datos relacionales, sistemas de archivos, red o servicios web de terceros.

1. Gestión de Pruebas con Apache Maven

En el ecosistema Java/Spring Boot, **Apache Maven** orquesta la gestión de dependencias y el ciclo de vida de la construcción del software (*Build Lifecycle*).

La Dependencia `spring-boot-starter-test`: En el archivo `pom.xml`, Spring Boot incluye el paquete acumulativo `spring-boot-starter-test` bajo el ámbito `<scope>test</scope>`. Este *starter* empaqueta automáticamente las librerías estándar del mercado: JUnit Jupiter,

Mockito, AssertJ, Hamcrest y Spring Test.

Ejecución con maven-surefire-plugin: Durante la fase `test` del ciclo de vida de Maven (`mvn test` o `mvn clean test`), el plugin `maven-surefire-plugin` escanea el proyecto, ejecuta automáticamente todas las clases de prueba cuyo nombre comience o termine en `Test` o `Tests` (ejemplo: `VehiculoServiceTest.java`) y genera reportes de ejecución en formato XML y texto dentro de `target/surefire-reports/`.

```
<!-- Fragmento de pom.xml para pruebas unitarias -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
```

2. Arquitectura de JUnit 5 y el Modelo JUnit Jupiter

A diferencia de versiones anteriores monolíticas (JUnit 4), **JUnit 5** está diseñado de forma modular y se compone de tres subproyectos independientes:

$$\text{JUnit 5} = \text{JUnit Platform} + \text{JUnit Jupiter} + \text{JUnit Vintage}$$

- **JUnit Platform:** Sirve como la base o infraestructura para lanzar marcos de prueba en la JVM. Define la interfaz de API entre IDEs (como Visual Studio Code, IntelliJ, Eclipse), herramientas de construcción (Maven, Gradle) y el motor de ejecución.
- **JUnit Jupiter:** Es la combinación del nuevo modelo de programación y modelo de extensión para escribir pruebas en JUnit 5. Introduce anotaciones avanzadas, pruebas parametrizadas y un motor nativo (`TestEngine`).
- **JUnit Vintage:** Proporciona un `TestEngine` de compatibilidad para ejecutar pruebas antiguas escritas en JUnit 3 y JUnit 4 dentro de la infraestructura de JUnit 5.

Anotaciones Fundamentales de JUnit Jupiter

- **@Test:** Denota que un método es un método de prueba.
- **@BeforeEach / @AfterEach:** Ejecuta el método anotado antes o después de **cada** método de prueba individual dentro de la clase (ideal para reiniciar estados y *mocks*).

- **@BeforeAll / @AfterAll:** Ejecuta el método una única vez antes o después de **todas** las pruebas de la clase (debe ser un método estático `static`).
- **@DisplayName("..."):** Declara un nombre descriptivo e intuitivo para la prueba en los reportes de Maven o del IDE.
- **@Disabled:** Desactiva temporalmente una prueba sin necesidad de borrar el código.
- **@ParameterizedTest:** Permite ejecutar la misma prueba múltiples veces pasándole diferentes conjuntos de datos de entrada mediante proveedores como `@ValueSource`, `@CsvSource` o `@MethodSource`.

3. La Estructura Analítica AAA (Arrange - Act - Assert)

Todo método de prueba unitaria debe estructurarse siguiendo el patrón AAA para garantizar legibilidad, mantenibilidad y objetividad:

1. **Arrange (Preparar / Given):** Se configuran las condiciones iniciales del entorno, se instancian los objetos requeridos y se programa el comportamiento esperado de los componentes simulados (*mocks*) mediante directivas como `when(...).thenReturn(...)`.
2. **Act (Ejecutar / When):** Se invoca explícitamente el método o funcionalidad bajo prueba (*System Under Test - SUT*).
3. **Assert (Verificar / Then):** Se valida formalmente que el resultado devuelto por el método coincida con los valores esperados (`assertEquals`, `assertTrue`) o que se hayan lanzado las excepciones de negocio correspondientes (`assertThrows`). Adicionalmente, se verifica que los *mocks* hayan sido invocados con las frecuencias y argumentos esperados (`verify(...)`).

4. Dobles de Prueba con Mockito en Spring Boot

Cuando se prueba una clase de negocio (ejemplo: `vehiculoService`), esta depende de interfaces de datos (ejemplo: `vehiculoRepository`). En una prueba unitaria pura, **no se debe conectar a una base de datos real**. En su lugar, se emplean *mocks* creados por Mockito:

- **@ExtendWith(MockitoExtension.class):** Anotación de JUnit Jupiter que integra el ciclo de vida de Mockito sin necesidad de cargar el contexto completo de Spring.
- **@Mock:** Crea una instancia simulada de una dependencia.

- **@InjectMocks:** Instancia automáticamente la clase bajo prueba e inyecta dentro de ella los campos anotados con @Mock.
- **MockMvc y @WebMvcTest:** Permiten realizar *Slice Testing* (pruebas de rebanada) en la capa de controladores REST (@RestController), evaluando serializaciones JSON y estados HTTP sin levantar un servidor de aplicaciones Tomcat.

Analogía (La Intuición)

Imagine la evaluación de seguridad de un motor de aviación comercial en una planta de ensamblaje:

- **Ciclo de Construcción con Maven (Línea de Ensamblaje):** Maven actúa como la cinta transportadora y el capataz de la fábrica. Cuando usted ordena `mvn test`, el capataz toma las piezas producidas, convoca a los inspectores de calidad (**JUnit Jupiter**) y les exige evaluar cada módulo antes de empaquetar el avión final.
- **JUnit Jupiter (El Kit de Herramientas de Inspección):** Es la maleta calibrada de instrumentos que usa el inspector. Contiene cronómetros (`assertTimeout`), calibradores (`assertEquals`) y etiquetas dinámicas (`@DisplayName`) para certificar que el motor cumple las tolerancias exigidas.
- **Mockito (Los Simuladores de Flujo):** En lugar de montar el motor en el avión completo y volar hacia el océano para probarlo (prueba E2E costosa y peligrosa), el motor se coloca individualmente en un banco de pruebas estático (*SUT*). Mockito sustituye los tanques de combustible y los controles de cabina por sensores electrónicos simulados (*mocks*). Si la computadora inyecta una señal de aceleración simulada, los instrumentos de JUnit Jupiter miden la respuesta en 50 milisegundos sin consumir una sola gota de combustible real.

Ejemplo de Código e Implementación (El 'Cómo')

A continuación se presenta la implementación de pruebas avanzadas combinando Maven, JUnit Jupiter y Mockito:

1. Prueba Unitaria con JUnit Jupiter Parametrizado y Mockito (VehiculoServiceTest)

```
package cr.ac.ucr.paraíso.ie.expresofast.business;

import cr.ac.ucr.paraíso.ie.expresofast.data.VehiculoRepository;
import cr.ac.ucr.paraíso.ie.expresofast.domain.Vehiculo;
import cr.ac.ucr.paraíso.ie.expresofast.exception.RecursoNoEncontradoException;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
@DisplayName("Pruebas Unitarias de la Capa de Servicio de Vehículos")
class VehiculoServiceTest {

    @Mock
    private VehiculoRepository vehiculoRepository;

    @InjectMocks
    private VehiculoService vehiculoService;

    private Vehiculo vehiculoPrueba;

    @BeforeEach
    void setUp() {
        vehiculoPrueba = new Vehiculo();
        vehiculoPrueba.setId(1L);
        vehiculoPrueba.setPlaca("SJB-9876");
        vehiculoPrueba.setModelo("Freightliner M2");
        vehiculoPrueba.setCapacidadCargaKg(5000.0);
    }
}
```

```

@Test
@DisplayName("Debe buscar y retornar un vehículo exitosamente cuando el ID existe")
void obtenerPorId_IdExistente_ReturnaVehiculoDto() {
    // Arrange (Given)
    when(vehiculoRepository.findById(1L)).thenReturn(Optional.of(vehiculoPrueba));

    // Act (When)
    Vehiculo resultado = vehiculoService.obtenerPorId(1L);

    // Assert (Then)
    assertAll("Verificaciones del Vehículo Retornado",
        () -> assertNotNull(resultado, "El vehículo retornado no debe ser nulo"),
        () -> assertEquals(1L, resultado.getId()),
        () -> assertEquals("SJB-9876", resultado.getPlaca())
    );

    // Verificación de comportamiento en el mock
    verify(vehiculoRepository, times(1)).findById(1L);
}

@ParameterizedTest
@ValueSource(longs = {99L, 100L, 500L})
@DisplayName("Debe lanzar RecursoNoEncontradoException para múltiples IDs inexistentes")
void obtenerPorId_IdInexistente_LanzaExcepcion(Long idInexistente) {
    // Arrange (Given)
    when(vehiculoRepository.findById(idInexistente)).thenReturn(Optional.empty());

    // Act & Assert (When & Then)
    RecursoNoEncontradoException excepcion = assertThrows(
        RecursoNoEncontradoException.class,
        () -> vehiculoService.obtenerPorId(idInexistente),
        "Debe lanzar la excepción cuando no se encuentra el registro"
    );

    assertTrue(excepcion.getMessage().contains(String.valueOf(idInexistente)));
    verify(vehiculoRepository, times(1)).findById(idInexistente);
    verify(vehiculoRepository, never()).save(any());
}
}

```

2. Prueba Unitaria de Rebanada en Controlador REST (VehiculoControllerTest)

```

package cr.ac.ucr.paraíso.ie.expresofast.controller;

```

```

import cr.ac.ucr.paraíso.ie.expresofast.business.VehiculoService;
import cr.ac.ucr.paraíso.ie.expresofast.domain.Vehiculo;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;

import static org.mockito.BDDMockito.given;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@WebMvcTest(VehiculoController.class)
@DisplayName("Pruebas de Rebanada en Controlador REST de Vehículos")
class VehiculoControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private VehiculoService vehiculoService;

    @Test
    @DisplayName("GET /api/v1/vehiculos/1 debe retornar HTTP 200 y la carga JSON esperada")
    void obtenerPorId_EndpointResponde200YJsonCorrecto() throws Exception {
        // Arrange
        Vehiculo v = new Vehiculo();
        v.setId(1L);
        v.setPlaca("SJB-9876");

        given(vehiculoService.obtenerPorId(1L)).willReturn(v);

        // Act & Assert
        mockMvc.perform(get("/api/v1/vehiculos/1")
                        .accept(MediaType.APPLICATION_JSON))
                .andExpect(status().isOk())
                .andExpect(content().contentType(MediaType.APPLICATION_JSON))
                .andExpect(jsonPath("$.id").value(1))
                .andExpect(jsonPath("$.placa").value("SJB-9876"));
    }
}

```

Explicación Paso a Paso del Código:

1. **@ParameterizedTest y @ValueSource**: Característica nativa de JUnit Jupiter. Ejecuta el mismo método de prueba 3 veces consecutivas, inyectando los valores 99L, 100L y 500L como argumento `idInexistente`, evitando duplicar código de prueba.
2. **assertAll(...)**: Agrupa múltiples aserciones de JUnit Jupiter. A diferencia de invocar varios `assertEquals` seguidos (donde si el primero falla se detiene la ejecución), `assertAll` evalúa **todas** las aserciones agrupadas y reporta cuáles fallaron simultáneamente.
3. **@DisplayName**: Proporciona nombres legibles que sustituyen los nombres técnicos de los métodos en la consola del desarrollador o en la integración con Maven (`mvn test`).
4. **@ExtendWith(MockitoExtension.class)**: Inicializa los objetos `@Mock` e `@InjectMocks` sin requerir el arranque del contenedor de Spring Boot.
5. **@WebMvcTest y MockMvc**: Ejecuta un test de rebanada centrado exclusivamente en el transporte REST HTTP y serialización Jackson.

Trampas Comunes

Omisión del Ámbito `<scope>test</scope>` en Maven:

- *Error*: Declarar librerías de prueba como `mockito-core` sin especificar el ámbito `test` en el `pom.xml`, provocando que las dependencias de prueba se empaqueten dentro del archivo ejecutable `.jar` de producción.
- *Solución*: Asegurar que todas las librerías de prueba se encuentren agrupadas en `spring-boot-starter-test` o tengan explícitamente `<scope>test</scope>`.

Mezclar Anotaciones de JUnit 4 y JUnit Jupiter:

- *Error*: Importar `@Test` del paquete antiguo `org.junit.Test` en lugar del paquete de JUnit Jupiter `org.junit.jupiter.api.Test`, causando que las pruebas sean ignoradas o requieran el motor JUnit Vintage.
- *Solución*: Verificar que todas las importaciones del proyecto provengan estrictamente de `org.junit.jupiter.api.*`.

Uso Indiscriminado de `@SpringBootTest` para Pruebas Unitarias:

- *Error:* Anotar una prueba unitaria de la capa de servicio con `@SpringBootTest`, obligando al framework a cargar el contexto de Spring, conectarse a la base de datos y escanear componentes.
- *Solución:* Utilizar únicamente `@ExtendWith(MockitoExtension.class)` para mantener la ejecución en milisegundos.

Pregunta de Reflexión

Dado que Maven ejecuta la fase `test` de manera automática al compilar el proyecto (`mvn clean package`):

¿Qué ventajas operativas y de calidad aporta el uso de pruebas parametrizadas con JUnit Jupiter (`@ParameterizedTest`) combinadas con el plugin `maven-surefire-plugin` en un servidor de Integración Continua (CI/CD) cuando se requiere validar reglas de negocio complejas ante múltiples bordes de datos?